

Dynatrace Query Language (DQL) - Complete 2-Hour Workshop

Workshop Overview

Duration: 2 Hours

Target Audience: DevOps Engineers, SREs, Monitoring Teams

Prerequisites: Access to Dynatrace tenant

Objective: Master DQL for logs, entities, and timeseries data

Part 1: Fundamentals (30 minutes)

Module 1.1: Introduction to DQL (10 min)

What is DQL?

- Dynatrace Query Language for data exploration
- Used to query logs, metrics, entities, and events
- Pipe-based syntax (similar to Unix pipes)
- Real-time data analysis and visualization

Basic Structure:

```
fetch <data_source>  
| <command1>  
| <command2>  
| ...
```

Module 1.2: Basic Fetch Operations (20 min)

Exercise 1: Fetch All Logs

```
dql  
  
fetch logs
```

Explanation:

- `fetch logs` retrieves all log records
- Returns unlimited results (use carefully in production)
- Shows timestamp, loglevel, content, and associated entities

💡 **Best Practice:** Always use `limit` to control result size

Exercise 2: Limiting Results

```
dql  
  
fetch logs  
| limit 100
```

Explanation:

- `limit 100` restricts output to 100 records
- Improves query performance
- Essential for large datasets

🔧 **Try It:** Change limit to 10, 50, 500 and observe response time

Exercise 3: Exploring Host Entities

```
dql  
  
fetch dt.entity.host
```

Explanation:

- `dt.entity.host` retrieves all host entities
- Shows host metadata (name, ID, properties)
- Foundation for infrastructure monitoring

Key Entities in Dynatrace:

- `dt.entity.host` - Physical/virtual hosts
 - `dt.entity.process_group` - Application processes
 - `dt.entity.service` - Monitored services
-

Exercise 4: Exploring Process Groups

```
dql  
  
fetch dt.entity.process_group
```

Explanation:

- Retrieves all process group entities
 - Process groups represent running applications
 - Contains metadata like technology, name, and host relationships
-

Part 2: Filtering & Field Selection (30 minutes)

Module 2.1: Entity Filtering (15 min)

Exercise 5: Filter Specific Host

```
dql  
  
fetch dt.entity.host  
| filter id == "HOST-D00AF059376716EE"  
| fields ipAddress, osType, osArchitecture, entity.name
```

Explanation:

- `filter id == "HOST-D00AF059376716EE"` - Filters to specific host by ID
- `fields` - Selects only specified columns
- Shows: IP address, OS type, architecture, and hostname



Finding Entity IDs:

- Navigate to entity in Dynatrace UI
 - Copy from URL or entity properties
-

Exercise 6: Comprehensive Host Information

```
dql
```

```
fetch dt.entity.host
| filter id == "HOST-D00AF059376716EE"
| fields id,
    entity.name,
    ipAddress,
    osType,
    osVersion,
    cpuCores,
    physicalMemory,
    hypervisorType,
    cloudType,
    monitoringMode,
    state,
    tags
| limit 1
```

Explanation:

- Complete host profile query
- **Key fields:**
 - `cpuCores` - Number of CPU cores
 - `physicalMemory` - RAM in bytes
 - `hypervisorType` - VM platform (VMware, Hyper-V, etc.)
 - `cloudType` - Cloud provider (AWS, Azure, GCP)
 - `monitoringMode` - Full-stack vs infrastructure-only
 - `state` - Host status
 - `tags` - Custom labels

Module 2.2: Field Manipulation (15 min)

Exercise 7: Selective Field Query

```
dql

fetch dt.entity.host
| filter id == "HOST-D00AF059376716EE"
| fields id, ipAddress, osType, cpuCores
```

Explanation:

- Minimalist query for specific information
 - Faster execution with fewer fields
 - Useful for dashboards and reports
-

Exercise 8: Calculated Fields

```
dql

fetch dt.entity.host
| filter id == "HOST-D00AF059376716EE"
| fields id, entity.name, physicalMemory, cpuCores
| fieldsAdd memoryGB = physicalMemory / 1024 / 1024
```

Explanation:

- `fieldsAdd` creates new calculated columns
- `memoryGB = physicalMemory / 1024 / 1024` converts bytes to megabytes
 - First `/1024` - bytes to kilobytes
 - Second `/1024` - kilobytes to megabytes
- Original fields remain in output



Common Conversions:

- Bytes to GB: `/ 1024 / 1024 / 1024`
 - Milliseconds to seconds: `/ 1000`
 - Percentage formatting: `* 100`
-

Part 3: Timeseries Metrics (30 minutes)

Module 3.1: Basic Timeseries (10 min)

Exercise 9: Average CPU Usage

```
dql

timeseries avgCpu = avg(dt.host.cpu.usage)
```

Explanation:

- `timeseries` - Query time-series metrics

- `avg()` - Calculates average value
- `dt.host.cpu.usage` - CPU utilization metric (0-100%)
- Aggregates across ALL hosts

 **Visualization:** Creates line chart over time

Exercise 10: Host Availability

```
dql
timeseries avg(dt.host.availability)
```

Explanation:

- Monitors host uptime/availability
 - Values: 0 (down) to 1 (up)
 - Useful for SLA reporting
-

Module 3.2: Advanced Timeseries (20 min)

Exercise 11: Per-Host CPU (Incorrect Method)

```
dql
timeseries avgcpu = avg(dt.host.cpu.usage), by: {dt.entity.host}
| fieldsAdd hostName = dt.entity.host
```

Explanation:

- `by: {dt.entity.host}` - Splits data by each host
 - **✗ Issue:** Shows entity ID, not readable name
 - Multiple series for each host
-

Exercise 12: Per-Host CPU (Correct Method)

```
dql
timeseries avgcpu = avg(dt.host.cpu.usage), by: {dt.entity.host}
| fieldsAdd hostName = entityName(dt.entity.host)
```

- `entityName()` function converts ID to human-readable name
-  **Better:** Shows actual hostname in results
- Essential for reporting and dashboards

 **Key Function:** `entityName()` - Always use for entity IDs

Exercise 13: Memory Usage by Host

```
dql

timeseries avgmem = avg(dt.host.memory.usage), by: {dt.entity.host}
| fieldsAdd hostName = entityName(dt.entity.host)
```

Explanation:

- Monitors memory utilization per host
- `dt.host.memory.usage` - Memory percentage (0-100%)
- Pattern same as CPU query

 **Use Case:** Identify memory-constrained hosts

Exercise 14: Process CPU Usage

```
dql

timeseries usage = avg(dt.process.cpu.usage,
  filter: { dt.entity.process_group_instance == "PROCESS_GROUP_INSTANCE-538CC3B42212A65A" },
  by: { dt.entity.process_group_instance }
| fieldsAdd entityName(dt.entity.process_group_instance)
| limit 100
```

Explanation:

- `dt.process.cpu.usage` - Process-level CPU metric
- **Inline filter:** Filters specific process instance
- `by: { dt.entity.process_group_instance }` - Groups by process
- Monitors individual application CPU consumption

 **Use Case:** Troubleshoot high-CPU applications

Exercise 15: Overall Process Memory

```
dql

timeseries memusage = avg(dt.process.memory.working_set_size)
```

Explanation:

- `working_set_size` - Active memory used by processes
 - Measured in bytes
 - Aggregated across all processes
-

Exercise 16: Top 10 Memory Consumers

```
dql

timeseries working_set_size = avg(dt.process.memory.working_set_size,
  filter: { in(dt.entity.process_group_instance,
    classicEntitySelector("type(PROCESS_GROUP_INSTANCE) AND fromRelationship.isProcessOf(type(HOST),entityId(
      HOST-D00AF059376716EE)
    )),
  by: { dt.entity.process_group_instance }
| fieldsAdd entityName(dt.entity.process_group_instance)
| sort arraySum(working_set_size) desc
| limit 10
```

Explanation:

- **Complex filter breakdown:**
 - `in()` - Checks membership in list
 - `classicEntitySelector()` - Legacy entity selector syntax
 - `type(PROCESS_GROUP_INSTANCE)` - Filter to processes
 - `fromRelationship.isProcessOf()` - Relationship filter
 - `entityId(HOST-D00AF059376716EE)` - Specific host
- `arraySum(working_set_size)` - Sums timeseries values
- `sort ... desc` - Descending order (highest first)
- `limit 10` - Top 10 consumers

 **Critical Query:** Identify memory leaks and resource hogs

Part 4: Advanced Log Analysis (30 minutes)

Module 4.1: Basic Log Filtering (10 min)

Exercise 17: INFO Logs

```
dql

fetch logs
| filter loglevel == "INFO"
| limit 1000
```

Explanation:

- Filters by log level
 - Common levels: DEBUG, INFO, WARN, ERROR, FATAL
 - Returns up to 1000 INFO entries
-

Exercise 18: Host-Specific Logs

```
dql

fetch logs
| filter matchesValue(dt.entity.host, "HOST-D00AF059376716EE")
| filter loglevel == "INFO"
```

Explanation:

- `matchesValue()` - Matches entity relationship
 - Combines host filter + log level
 - Multiple filters combine with AND logic
-

Exercise 19: Error and Warning Logs

```
dql

fetch logs
| filter loglevel == "ERROR" or loglevel == "WARN"
| limit 1000
```

Explanation:

- `or` operator for multiple conditions
- Critical for incident investigation
- Focuses on problem indicators

 **Alert Use Case:** Create alerts on error count spikes

Module 4.2: Time-Based Log Queries (10 min)

Exercise 20: Last Hour's INFO Logs

```
dql

fetch logs, from: now() - 1h
| filter loglevel == "INFO"
| fields timestamp, loglevel, content, dt.entity.host
| sort timestamp desc
| limit 100
```

Explanation:

- `from: now() - 1h` - Last 60 minutes
 - `fields` - Selects specific columns
 - `sort timestamp desc` - Newest first
 - **Time syntax:**
 - `now()` - Current time
 - `-1h` - 1 hour ago
 - Other units: `m` (minutes), `d` (days), `w` (weeks)
-

Exercise 21: Enriched Log Data

```
dql
```

```
fetch logs
| filter loglevel == "INFO"
| fields timestamp, loglevel, content, dt.entity.host, dt.entity.process_group_instance
| fieldsAdd
  hostName = entityName(dt.entity.host),
  processName = entityName(dt.entity.process_group_instance)
| sort timestamp desc
| limit 100
```

Explanation:

- Shows both entity IDs and names
- `fieldsAdd` converts IDs to readable names
- Complete context for troubleshooting
- **Benefit:** No need to look up entity names separately

Module 4.3: Log Analytics (10 min)

Exercise 22: Log Level Distribution

```
dql

fetch logs, from: now() - 24h
| filter loglevel == "INFO"
| summarize count(), by: { loglevel }
| sort `count()` desc
```

Explanation:

- `from: now() - 24h` - Last 24 hours
- `summarize count()` - Aggregates record count
- `by: { loglevel }` - Groups by log level
- **Note:** Backticks around `count()` needed in sort



Dashboard Widget: Shows log volume by severity

Exercise 23: Content Search

```
dql
```

```
fetch logs
| filter (loglevel == "INFO")
    and matchesPhrase(content, "root")
| fields timestamp, loglevel, content
| sort timestamp desc
| limit 100
```

Explanation:

- `matchesPhrase()` - Full-text search in content
- Searches for "root" keyword
- Case-sensitive by default
- Useful for searching error messages, usernames, IDs

Use Cases:

- Find logs mentioning specific users
- Search for error codes
- Locate transaction IDs

Workshop Activities & Exercises

Hands-On Challenge 1: Host Health Dashboard (15 min)

Task: Create queries for a host monitoring dashboard

1. Find top 3 hosts by CPU usage
2. Find hosts with memory > 80%
3. Count ERROR logs per host in last hour

Solution Hints:

- Use `by: {dt.entity.host}` for grouping
- Combine `filter` with percentage thresholds
- Use `summarize count()` for log counts

Hands-On Challenge 2: Application Troubleshooting (15 min)

Scenario: Application is slow on specific host

Tasks:

- 1. Find all process groups on the host
- 2. Get CPU usage for each process
- 3. Find ERROR logs from the problematic process
- 4. Identify peak usage time

Quick Reference Card

Essential Commands

Command	Purpose	Example
<code>fetch</code>	Retrieve data	<code>fetch logs</code>
<code>filter</code>	Filter results	<code>filter loglevel == "ERROR"</code>
<code>fields</code>	Select columns	<code>fields timestamp, content</code>
<code>fieldsAdd</code>	Add calculated field	<code>fieldsAdd gb = bytes / 1024^3</code>
<code>limit</code>	Restrict count	<code>limit 100</code>
<code>sort</code>	Order results	<code>sort timestamp desc</code>
<code>summarize</code>	Aggregate data	<code>summarize count()</code>
<code>timeseries</code>	Time-series metrics	<code>timeseries avg(cpu)</code>

Key Functions

Function	Purpose	Example
<code>entityName()</code>	Get entity name	<code>entityName(dt.entity.host)</code>
<code>avg()</code>	Average value	<code>avg(dt.host.cpu.usage)</code>
<code>count()</code>	Count records	<code>count()</code>
<code>sum()</code>	Sum values	<code>sum(bytes)</code>
<code>matchesPhrase()</code>	Text search	<code>matchesPhrase(content, "error")</code>
<code>matchesValue()</code>	Entity match	<code>matchesValue(dt.entity.host, "HOST-123")</code>

Time Syntax

- `now()` - Current time
- `now() - 1h` - 1 hour ago
- `now() - 30m` - 30 minutes ago
- `now() - 1d` - 1 day ago

- `from: timestamp, to: timestamp` - Custom range
-

Best Practices

Performance

1.  Always use `limit` on large datasets
2.  Filter early in query pipeline
3.  Use specific time ranges
4.  Avoid `fetch logs` without filters in production

Readability

1.  Use `entityName()` for human-readable output
2.  Add descriptive aliases: `avgCpu = avg(cpu)`
3.  Format multi-line queries with proper indentation
4.  Comment complex queries

Monitoring

1.  Combine metrics with logs for context
 2.  Use `by:` clause for granular analysis
 3.  Create reusable query templates
 4.  Document query purpose and usage
-

Common Patterns

Pattern 1: Host Deep Dive

```
dql

fetch dt.entity.host
| filter id == "YOUR-HOST-ID"
| fields entity.name, cpuCores, physicalMemory
| fieldsAdd memoryGB = physicalMemory / 1024 / 1024 / 1024
```

Pattern 2: Error Investigation

```
dql
```

```
fetch logs, from: now() - 1h
| filter loglevel == "ERROR"
| fields timestamp, content, dt.entity.host
| fieldsAdd hostName = entityName(dt.entity.host)
| sort timestamp desc
```

Pattern 3: Resource Monitoring

```
dql

timeseries avg(dt.host.cpu.usage), by: {dt.entity.host}
| fieldsAdd hostName = entityName(dt.entity.host)
| filter avgCpu > 80
```

Next Steps

Advanced Topics

1. **DQL Functions** - String manipulation, math operations
2. **Joins** - Combine multiple data sources
3. **Alerting** - Create alerts from queries
4. **Dashboards** - Build custom visualizations
5. **DQL API** - Programmatic query execution

Resources

- Dynatrace Documentation: Query logs and entities
- Dynatrace Community Forum
- DQL Playground in Dynatrace UI (Notebooks)

Workshop Summary

You've learned to:  Fetch logs and entities

-  Filter and select specific data
-  Create calculated fields
-  Query timeseries metrics
-  Analyze logs with time ranges
-  Perform aggregations and grouping

- ✓ Convert entity IDs to names
- ✓ Build production-ready queries

Key Takeaway: DQL is your Swiss Army knife for Dynatrace data exploration!

Q&A and Feedback

Thank you for attending! 🎉

Practice Assignment: Create a dashboard showing:

1. Top 5 hosts by CPU
2. Error log count (last 24h)
3. Memory usage trend
4. Process with highest memory consumption